Going Deep

Jonathon Hare

Overview

- No free lunch and universal approximation
- Why go deep?
- Problems of going deep
- Some fixes:
  - Improving gradient flow with skip connections
  - Regularising with Dropout

No Free Lunch

- Statistical learning theory claims that a machine can generalise well from a finite training set.
- This contradicts basic inductive reasoning which says to derive a rule describing every member of a set one must have information about every member.
- Machine learning avoids this problem by learning probabilistic[1] rules which are *probably* correct about *most* members of the set they concern.
- But, **no free lunch theorem** states that every possible classification machine has the *same error* when averaged over *all possible* data-generating distributions.
  - **No machine learning algorithm is universally better than any other!**
  - Fortunately, in the real world, data is generated by a small subset of generating distributions...

pause

The Universal Approximation Theorem

Let $\psi : \mathbb{R} \to \mathbb{R}$ be a nonconstant, bounded, and continuous function. Let I_m denote the m-dimensional unit hypercube $[0, 1]^m$. The space of real-valued continuous functions on I_m is denoted by $C(I_m)$. Then, given any $\varepsilon > 0$ and any function $f \in C(I_m)$, there exists an integer N, real constants $v_i, b_i \in \mathbb{R}$ and real vectors $w_i \in \mathbb{R}^m$ for $i = 1, \ldots, N$, such that we may define:

$F(x) = \sum_{i=1}^{N} v_i \psi(w_i^T x + b_i)$ as an approximate realization of the function f ; that is,

$|F(x) - f(x)| < \varepsilon \ \forall \ x \in I_m$. [1em] $\implies$ simple neural networks can represent a wide variety of interesting functions when given appropriate parameters.

So a single hidden layer can approximate any continuous function?

- Yes!
- But, ...

[1] or perhaps more generally rules which are not certain

- to get the precision you require (small ε), you might need a really large number of hidden units (very large N).
- worse-case analysis shows it might be exponential (possibly one hidden unit for *every* input configuration)
- We've not said anything about learnability...
  * The optimiser might not find a good solution[2].
  * The training algorithm might just choose the wrong solution as a result of overfitting.
  * *There is no known universal procedure for examining a set of examples and choosing a function that will generalise to points out of the training set.*

Then Why Go Deep?

- There are functions you can compute with a deep neural network that shallow networks require exponentially more hidden units to compute.
  - The following function is more efficient to implement using a deep neural network: $y = x_1 \oplus x_2 \oplus x_3 \oplus \cdots \oplus x_n$
- We should care about the data generating distribution (c.f. NFL).
  - Real-world data has significant structure; often believed to be generated by (relatively) simple low-dimensional latent processes.
  - Implies a prior belief that the underlying factors of variation in data can be explained by a hierarchical composition of increasingly simple latent factors
- Alternatively, one could just consider that a deep architecture just expresses that the function we wish to learn is a program made of multiple steps where each step makes use of the previous steps outputs.
- **Empirically, deeper networks just seem to generalise better!**

What are the problems?

- Learnability is still hard
  - Problems of gradient flow
  - Horrible symmetries in the loss landscape
  - Overfitting

pause

Vanishing and Exploding Gradients

- The vanishing and exploding gradient problem is a difficulty found in training NN with gradient-based learning methods and backpropagation.
- In training, the gradient may become vanishingly small (or large), effectively preventing the weight from changing its value (or exploding in value).
- This leads to the neural network not being able to train.
- This issue affects many-layered networks (feed-forward), as well as recurrent networks.
- Adaptive optimisers can rescale small parameter gradients, but they cannot recover information lost through repeated poorly conditioned Jacobians.

pause

[2]note that it has been shown that the gradients of the function are approximated by the network to an arbitrary precision

Residual Connections

- One of the most effective ways to make very deep networks trainable is to use residual neural networks (ResNets)[3].
- ResNets are artificial neural networks that use *skip connections* to jump over layers.
- The skip path contributes an identity term to the backward Jacobian, providing a direct route for the gradient.
- This is not the full story: skip connections also change the optimisation geometry and symmetries.

pause

Residual Connections

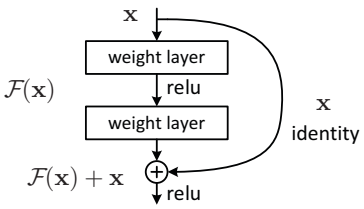


Figure 2. Residual learning: a building block. .

Residual Gradient Flow

A residual block computes

$$\boldsymbol{y} = \boldsymbol{x} + F(\boldsymbol{x}; \boldsymbol{\theta}) \, .$$

If $J_F(\boldsymbol{x})$ denotes the Jacobian of F evaluated at $\boldsymbol{x}$, then the Jacobian of the block is

$$\frac{\partial \boldsymbol{y}}{\partial \boldsymbol{x}} = I + J_F(\boldsymbol{x}).$$

For a scalar loss L and column-vector gradients, backprop computes $\nabla_{\boldsymbol{\theta}} L$ to update the parameters and $\nabla_{\boldsymbol{x}} L$ to propagate the gradient to earlier layers:

$$\nabla_{\boldsymbol{x}} L = \nabla_{\boldsymbol{y}} L + \underbrace{J_F(\boldsymbol{x})^T \nabla_{\boldsymbol{y}} L}_{\text{VJP through } F} \, .$$

- The gradient has a direct identity path as well as the path through F.
- This improves gradient flow, although it does not guarantee that gradients remain well scaled.

pause

Residual Connections

[3]K. He, X. Zhang, S. Ren and J. Sun, "Deep Residual Learning for Image Recognition," CVPR, Las Vegas, NV, 2016, pp. 770-778.

K. He, X. Zhang, S. Ren and J. Sun, "Deep Residual Learning for Image Recognition," CVPR, Las Vegas, NV, 2016, pp. 770-778.

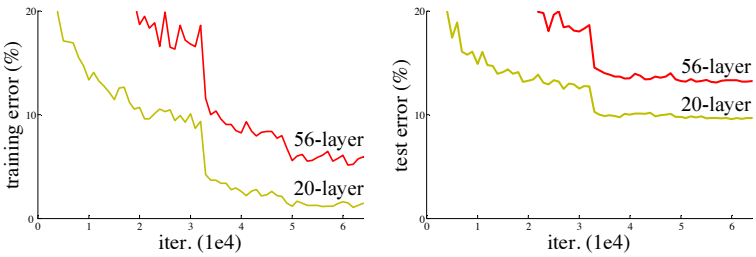


Figure 1. Training error (left) and test error (right) on CIFAR-10 with 20-layer and 56-layer "plain" networks. The deeper network has higher training error, and thus test error. Similar phenomena on ImageNet is presented in Fig. 4.

pause

Residual Connections

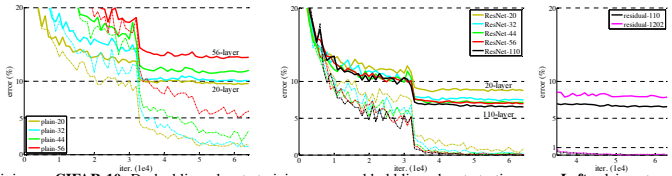


Figure 6. Training on **CIFAR-10**. Dashed lines denote training error, and bold lines denote testing error. **Left**: plain networks. The error of plain-110 is higher than 60% and not displayed. **Middle**: ResNets. **Right**: ResNets with 110 and 1202 layers.

Overfitting

- Neural networks with a large number of parameters (and hidden layers) are powerful, however, overfitting is a serious problem in such systems.
- Just as you've seen in simple machines (e.g. Ridge Regression and LASSO), regularisation can help mitigate overfitting
- In deep networks, we might:
  - Use the architecture to regularise (e.g. ConvNets)
  - Use weight regularisers (L1, L2 [weight decay], etc, ...)
  - Use a stochastic weight regulariser (like dropout)

pause

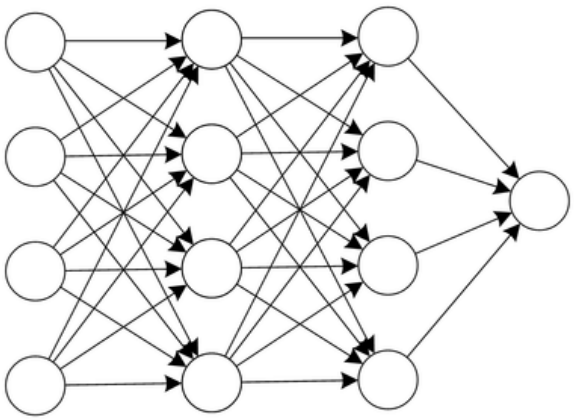
Dropout

- Dropout is a form of regularisation
- The key idea in dropout is to randomly drop neurons, including all of the connections, from the neural network during training.
- Motivation: averaging predictions from several models often improves generalisation.
  - Training many networks independently is expensive.
  - Dropout samples many overlapping subnetworks that share parameters.

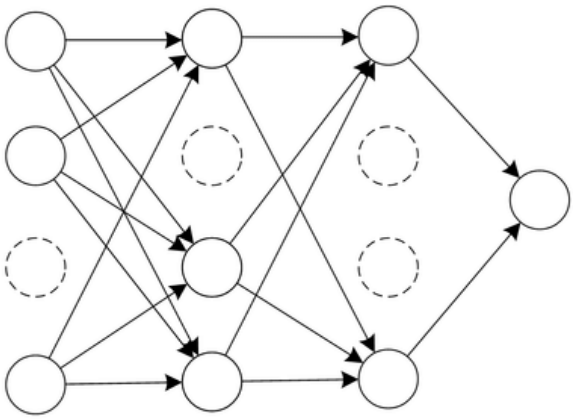
K. He, X. Zhang, S. Ren and J. Sun, "Deep Residual Learning for Image Recognition," CVPR, Las Vegas, NV, 2016, pp. 770-778.

K. He, X. Zhang, S. Ren and J. Sun, "Deep Residual Learning for Image Recognition," CVPR, Las Vegas, NV, 2016, pp. 770-778.

Dropout



(a) Standard Neural Network



(b) Network after Dropout

pause

How Does Dropout Work?

- In the learning phase, we set a dropout probability for each layer in the network.
- On each training forward pass, we sample a binary mask and apply it to the activations.
- **Inverted Dropout** rescales the retained activations so that their expected magnitude is unchanged when dropout is disabled at evaluation/test time.

pause

How is Inverted Dropout implemented?

- We define a random binary mask $\boldsymbol{m}^{(l)}$ for the activations in layer l. Each element is sampled from a Bernoulli distribution with $P(m_i^{(l)} = 1) = p$, where p is the *keep probability*. A new mask is sampled on each training forward pass.
- In `torch.nn.Dropout`, the argument `p` is instead the *drop probability*, so the keep probability is $1 - p$.
- The forward pass of a Dropout layer (function) during **training** is given by $f(\boldsymbol{x}) = \boldsymbol{x} \odot \boldsymbol{m}/p$.
- The forward pass of a Dropout layer (function) during **inference** is given by $f(\boldsymbol{x}) = \boldsymbol{x}$.
- This can be applied to any layer(s) of the network *except the output layer!*
- It's not common to put it everywhere; just a couple of select places (empirically chosen).
- The gradient (during training) is simply the hadamard product of the incoming gradient with $\boldsymbol{m}/p$.

pause

Why Does Dropout Work?

- A unit cannot rely on a fixed set of other units always being present.
- The noise discourages fragile co-adaptation between features.
- At inference, the full network approximates an average over many overlapping subnetworks.

Image from: https://www.researchgate.net/figure/Dropout-neural-network-model-a-is-a-standard-neural-network-b-is-the-same-network_fig3_309206911